



Sujet de Projet – Application Mobile “Weather Secure App”

Matière : React Native

Formation : Bachelor

Auteur : Malik HARRIZ

Date : 25/06/2026

1 Sujet de Projet – Application Mobile “Weather Secure App”

1.1 Objectif

Concevoir et développer une application mobile en **React Native** permettant d’interagir avec l’API **OpenWeather**, tout en intégrant :

- une authentification sécurisée avec **Firebase**,
- un stockage local avec **SQLite**,
- une validation avancée des formulaires avec **Joi**.

1.2 Description du projet

L’application doit permettre à un utilisateur de :

- créer un compte et se connecter de manière sécurisée,
- rechercher des informations météorologiques via OpenWeather <https://openweathermap.org/>,
- consulter des données détaillées,
- stocker localement certaines données (historique, favoris),
- interagir avec des formulaires validés et sécurisés.

1.3 Contraintes techniques

- Framework : React Native (Expo)
- Authentification : Firebase Authentication
- API : OpenWeather API
- Stockage local : SQLite
- Validation des formulaires : Joi (via une intégration adaptée)
- Navigation : React Navigation <https://reactnavigation.org/>
- Requêtes HTTP : Fetch

1.4 Fonctionnalités attendues

1.4.1 1. Authentification avec Firebase

1.4.1.1 Fonctionnalités :

- Inscription (email + mot de passe)
- Connexion
- Déconnexion
- Réinitialisation du mot de passe

1.4.1.2 Contraintes de sécurité :

- Validation avec Joi :
 - Email valide
 - Mot de passe fort (longueur minimale, majuscules, chiffres)
- Messages d'erreur explicites
- Protection des routes (navigation réservée aux utilisateurs authentifiés)
- Gestion sécurisée de la session utilisateur

1.4.2 Intégration de l'API OpenWeather

1.4.2.1 Fonctionnalités :

- Recherche d'une ville
- Affichage de la météo actuelle
- Affichage des détails :
 - Température
 - Humidité
 - Vitesse du vent
 - Conditions climatiques

1.4.2.2 Contraintes :

- Gestion des erreurs (ville inexistante, problème réseau)

- Indicateur de chargement
- Formatage des données pour l'affichage

1.4.3 Navigation et écrans (minimum 5 écrans)

L'application doit contenir au minimum les écrans suivants :

1. Écran Authentification

- Inscription / Connexion
- Validation via Joi

2. Écran Dashboard

- Météo d'une ville par défaut
- Accès rapide aux fonctionnalités

3. Écran Recherche

- Saisie d'une ville
- Affichage des résultats

4. Écran Détail météo

- Informations météo complètes

5. Écran Profil

- Informations utilisateur
- Déconnexion

1.4.4 Sécurité des formulaires avec Joi (exigence critique)

1.4.4.1 Exigences :

- Validation des données :
 - Schémas définis (email, password, champs texte)
- Vérification des champs :
 - Champs obligatoires
 - Formats valides
- Gestion des erreurs :
 - Messages explicites
- Protection contre :
 - données incohérentes
 - soumissions invalides

1.4.5 Stockage local avec SQLite

1.4.5.1 Fonctionnalités :

- Historique des recherches météo
- Sauvegarde des villes favorites

1.4.5.2 Contraintes :

- Organisation des données en tables
- Accès rapide aux données locales
- Synchronisation simple avec l'interface utilisateur

1.5 Livrables attendus

- Code source versionné (Git)
- README :
 - Installation
 - Configuration Firebase et OpenWeather
- Documentation sur :
 - la sécurité des formulaires (Joi)
 - la gestion des données (Firebase et SQLite)

1.6 Date & Modalité rendu

1. créer un dépôt github et réaliser un push de l'ensemble des éléments
2. remplir le formulaire disponible à l'adresse suivante :
3. <https://forms.gle/ozm9zWCZwmaAei9y7>
4. Date de rendu maximum : le 1er Juillet 2026 20h00

1.7 Critères d'évaluation

Critère	Pondération
Fonctionnalités	30%
Sécurité (Joi + Firebase)	25%
Qualité du code	20%
Expérience utilisateur	15%
Documentation	10%

1.8 Extensions possibles (bonus)

- Géolocalisation pour météo automatique
- Mode sombre
- Mise en cache améliorée
- Synchronisation entre Firebase et SQLite
- Ajout de graphiques météo